

Spring Boot 가이드

디자인 패턴 : 낯선 용어가 아니라, 이미 알고 있던 이야기

“프로젝트 코드에서 발견하는 디자인 패턴에 대한 이해”

※ 이 문서는 새로운 코드를 배우는 게 아니라, 가이드 1~3에서 이미 경험한 loginauth-board 프로젝트 코드를 다시 열어보며 "이게 사실 디자인 패턴이었구나"를 확인하는 문서입니다.

이 문서를 처음 여신 분이라면, "디자인 패턴"이라는 말부터 부담스러울 수 있습니다. 하지만 걱정하지 않으셔도 됩니다. 이 안에 나오는 다섯 가지는 전부 가이드 1~3에서 이미 코드로 직접 보셨던 것들입니다. 복사기, 카페, 여행 패키지, 비서, 공항 검문소처럼 우리가 일상에서 흔히 겪는 장면에 빗대어, 조금 더 편안하게 다시 만나보려 합니다.

가이드 1에서 필드 • import를 따라 구조를 읽었고, 가이드 2에서 요청 하나가 필터 → 컨트롤러 → 서비스를 거치는 흐름을 봤고, 가이드 3에서 post 도메인이 같은 구조를 반복하는 걸 확인했습니다. 이번 가이드는 그 코드들을 다시 꺼내, 왜 이런 모습으로 설계됐는지 "이름"을 붙여보는 시간입니다.

전체 지도 : 5 개 패턴, 5 개의 익숙한 이야기

패턴	일상 속 비유	코드 위치	카테고리
① 싱글톤	다 같이 쓰는 복사기 한 대	AuthService 등 @Service/@Repository/@Component 전체	생성
② 팩토리 메서드	주문만 하면 되는 바리스타	SecurityConfig.passwordEncoder()	생성
③ 템플릿 메서드	일정 틀이 고정된 패키지 여행	JdbcUserRepository (JdbcTemplate)	행동
④ 프록시	전화를 대신 받아주는 비서	AuthService(@Transactional) / PostRepository(JPA)	구조
⑤ 체인 오브 리스 폰서빌리티	순서대로 통과하는 공항 검문소	SecurityConfig + JwtAuthenticationFilter	행동

패턴 1. 싱글톤 (생성 패턴)

이야기로 먼저 생각해볼까요?

사무실에 복사기가 한 대 있습니다. 만약 부서마다 복사기를 따로 산다면 어떨까요? 관리도 번거롭고, 종이와 잉크도 낭비되고, 무엇보다 돈이 많이 듭니다. 그래서 보통은 한 층에 복사기를 딱 한 대만 두고, 모든 직원이 그 한 대를 함께 씁니다. AuthService도 이 복사기와 똑같은 역할입니다.

① 프로젝트 코드 위치

loginauth/auth/service/AuthService.java (그리고 @Repository, @Component가 붙은 모든 클래스)

② 해당 디자인 패턴

싱글톤 : 생성 패턴입니다. 방금 이야기한 "복사기 한 대를 다 같이 쓰는" 방식을 코드로 그대로 구현한 것이 바로 싱글톤 패턴입니다.

③ 코드 설명

```
@Service
public class AuthService {
    private final UserRepository userRepository;
    private final PasswordEncoder passwordEncoder;

    public AuthService(UserRepository userRepository, PasswordEncoder
passwordEncoder) {
        this.userRepository = userRepository;
        this.passwordEncoder = passwordEncoder;
    }
    ...
}
```

@Service 애노테이션이 붙으면, Spring은 서버가 시작될 때 이 AuthService를 딱 한 번만 만들어 컨테이너라는 창고에 넣어둡니다. 마치 층 전체가 함께 쓸 복사기를 한 대만 사서 복사실에 가져다 놓는 것과 같습니다. 이후 AuthController를 비롯해 이 객체가 필요한 모든 곳에는 창고에 있던 그 하나의 인스턴스가 그대로 전달됩니다.

코드 어디에도 new AuthService()를 직접 호출한 부분이 없는데도 정상적으로 동작하는 이유가 여기 있습니다. "복사기를 누가, 언제, 몇 대 살지"를 각 부서(개발자)가 알아서 정하는 게 아니라, 총무팀(Spring 컨테이너)이 한 번에 책임지고 관리해주는 셈입니다.

④ 왜 필요했고, 왜 의미 있었는가?

만약 정말로 매 요청마다 새 AuthService를 만든다면, 부서마다 복사기를 따로 사는 것과 다를 게 없습니다. 그 안에 딸려 있는 UserRepository나 PasswordEncoder 같은 부속품까지 매번 새로 사야 하니, 메모리와 시간이 그만큼 낭비됩니다.

특히 DB 연결처럼 만드는 데 비용이 큰 자원을 다루는 객체를 매번 새로 만드는 건, 복사기를 매번 새로 사는 것보다 훨씬 비싼 일입니다. 싱글톤은 "어차피 상태 없이 일만 하는 직원(객체)은 한 명이면 충분하다"는 원칙을 프레임워크가 대신 지켜주는 장치입니다.

패턴 2. 팩토리 메서드 (생성 패턴)

이야기로 먼저 생각해볼까요?

카페에서 "아메리카노 한 잔 주세요"라고 주문해본 적 있으실 겁니다. 이때 손님은 원두를 어떻게 갈고, 몇 도의 물로 몇 초 동안 내리는지 전혀 몰라도 됩니다. 그 과정은 전부 바리스타가 알아서 처리하고, 손님 손에는 완성된 커피 한 잔만 쥐어집니다. SecurityConfig의 passwordEncoder() 메서드가 바로 이 바리스타 역할을 합니다.

① 프로젝트 코드 위치

loginauth/global/config/SecurityConfig.java

② 해당 디자인 패턴

팩토리 메서드 : 생성 패턴입니다. "주문만 하면 만드는 과정은 몰라도 되는" 바리스타의 역할을 코드로 옮긴 것이 팩토리 메서드 패턴입니다.

③ 코드 설명

```
@Bean
public PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder(12);
}
```

이 메서드는 "BCryptPasswordEncoder라는 커피를 만들어주는 바리스타"입니다. @Bean이 붙어 있어서, Spring은 서버가 시작될 때 이 메서드를 딱 한 번 호출해 완성된 커피(객체)를 컨테이너에 올려둡니다.

중요한 건 손님 역할을 하는 AuthService 쪽입니다. 생성자를 보면 PasswordEncoder라는 이름표만 받을 뿐, 그게 정확히 BCrypt로 내린 커피인지 다른 방식인지는 전혀 알지 못합니다. "무

엇을, 어떻게 만들지 결정하는 코드" (바리스타)와 "그걸 받아서 쓰는 코드" (손님)가 완전히 분리되어 있는 것입니다.

④ 왜 필요했고, 왜 의미 있었는가?

나중에 카페가 원두를 바꾸듯, 암호화 방식을 BCrypt에서 다른 방식으로 바꿔야 한다면 passwordEncoder() 메서드의 return 한 줄만 고치면 됩니다. 손님 (AuthService, AuthController)은 카페 뒤에서 무슨 일이 있었는지 몰라도, 여전히 커피 한 잔을 받을 수 있습니다.

"만드는 방법"을 한 곳 (팩토리 메서드)에 몰아두면, 나중에 만드는 방식이 바뀌어도 영향 범위가 그 한 곳으로 좁혀진다는 것이 팩토리 메서드 패턴이 주는 가장 큰 안심입니다.

패턴 3. 템플릿 메서드 (행동 패턴)

이야기로 먼저 생각해볼까요?

패키지 여행을 가면 일정표가 항상 비슷한 틀을 가지고 있습니다. "공항 픽업 → 호텔 체크인 → 관광 → 저녁 식사"라는 순서는 어느 여행사를 가든 똑같습니다. 여행마다 바뀌는 건 딱 하나, "오늘은 어디를 구경하느냐"는 세부 코스뿐입니다. JdbcUserRepository가 하는 일도 이 여행사와 닮아 있습니다.

① 프로젝트 코드 위치

loginauth/auth/repository/JdbcUserRepository.java

② 해당 디자인 패턴

템플릿 메서드 : 행동 패턴입니다. "전체 일정 틀은 고정하고, 세부 코스만 갈아끼우는" 여행 패키지 상품의 원리를 코드로 옮긴 것이 템플릿 메서드 패턴입니다.

③ 코드 설명

```
@Override
public Optional<User> findByUsername(String username) {
    return jdbcTemplate.query(
        "SELECT id, username, password FROM users WHERE username = ?",
        (rs, rowNum) -> new User(rs.getLong("id"), rs.getString("username"),
rs.getString("password")),
        username
    ).stream().findFirst();
}
```

jdbcTemplate.query()를 호출하면, "커넥션을 얻고 → SQL을 실행하고 → 결과를 한 행씩 순회하고 → 자원을 정리하는" 정해진 일정(알고리즘)은 JdbcTemplate이라는 여행사가 전부 대신 처리해 줍니다. 우리가 실제로 챙긴 건 딱 하나, "오늘 방문할 곳"에 해당하는 램다식, 즉 "한 행(row)을 어떤 객체로 바꿀지"뿐입니다.

이 램다가 바로 여행 일정표의 빈 칸, "오늘 관광지" 자리에 끼워 넣는 내용입니다. 전체 일정(알고리즘) 뼈대는 고정하고, 달라지는 부분만 외부에서 주입받는 구조라 템플릿 메서드 패턴이라고 부릅니다.

④ 왜 필요했고, 왜 의미 있었는가?

템플릿 메서드가 없다면, findByUsername()을 만들 때마다 "커넥션 열기 → 실행 → 자원 정리"까지 전부 직접 짜야 합니다. 이건 여행사가 매번 항공권부터 호텔 예약까지 혼자 다 처리하는 것과 같습니다. 이 과정에서 커넥션을 닫는 걸 깜빡하는 실수, 즉 "호텔 체크아웃을 깜빡하는 일"이 실제로 자주 벌어집니다.

JdbcTemplate은 "누구나 실수할 수 있는 반복 일정"은 여행사가 가져가고, "이 프로젝트만 아는 고유한 취향 (행 매핑)"만 개발자에게 맡깁니다. 이게 템플릿 메서드 패턴이 실무에서 이렇게 널리 쓰이는 이유입니다.

패턴 4. 프록시 (구조 패턴)

이야기로 먼저 생각해볼까요?

바쁜 사장님에게 걸려오는 전화를 상상해보세요. 모르는 번호로 걸려오는 전화까지 사장님이 전부 직접 받는다면, 정작 중요한 업무를 할 시간이 없을 겁니다. 그래서 비서가 먼저 전화를 받아 용건을 확인하고, 정말 필요한 전화만 사장님께 연결해줍니다. AuthService의 @Transactional과 PostRepository는 이 비서 역할을 하는 코드입니다.

① 프로젝트 코드 위치

loginauth/auth/service/AuthService.java (@Transactional) 그리고

loginauth/post/repository/PostRepository.java

② 해당 디자인 패턴

프록시 : 구조 패턴입니다. "진짜 담당자 앞에 대리인을 세워, 먼저 확인하고 처리한 뒤에 연결

하는" 비서의 역할을 코드로 옮긴 것이 프록시 패턴입니다.

③ 코드 설명

```
@Transactional
public User signUp(String username, String rawPassword) {
    userRepository.findByUsername(username).ifPresent(user -> {
        throw new DuplicateUsernameException(username);
    });
    return userRepository.save(username, passwordEncoder.encode(rawPassword));
}

public interface PostRepository extends JpaRepository<Post, Long> {
    List<Post> findAllByOrderByIdDesc();
}
```

AuthController가 signUp()을 호출할 때, 사실 진짜 AuthService가 바로 전화를 받는 게 아닙니다. Spring이 미리 세워둔 "비서" (프록시)가 먼저 전화를 가로칩니다. 이 비서가 "지금부터 통화 시작합니다" (트랜잭션 시작)를 알리고, 그다음 진짜 AuthService.signUp()을 연결시켜주고, 통화가 무사히 끝나면 "통화 종료, 기록 저장" (커밋), 중간에 문제가 생기면 "통화 없었던 걸로 처리" (롤백)까지 대신 해줍니다.

두 번째 비서는 PostRepository입니다. JpaRepository만 상속했을 뿐 실제로 일하는 직원(구현 클래스)은 프로젝트 어디에도 없습니다. 서버가 문을 열면 Spring Data JPA가 이 자리에 "대리 직원"(프록시)을 알아서 채용해 얹혀둡니다. save(), findById() 같은 요청이 오면 이 대리 직원이 SQL을 만들어 대신 처리합니다.

④ 왜 필요했고, 왜 의미 있었는가?

트랜잭션 처리 (통화 시작·종료·취소)나 기본 CRUD SQL 실행은 반복적이면서도, 한 번만 실수해도 데이터가 깨질 수 있는 위험한 업무입니다. 이런 업무를 개발자가 직접 하지 않고 비서 (프록시)에게 맡기면, 개발자는 "무슨 용건으로 통화할지 (비즈니스 로직)"만 준비하면 되고, "어떻게 안전하게 연결할지"는 비서가 알아서 책임집니다.

@Transactional 하나, extends JpaRepository 한 줄 뒤에 이렇게 성실한 비서가 일하고 있다는 걸 알면, 앞으로 이 두 코드를 볼 때마다 "지금 대리인이 대신 일하고 있구나"가 자연스럽게 떠오르실 겁니다.

패턴 5. 체인 오브 리스폰서빌리티 (행동 패턴)

이야기로 먼저 생각해볼까요?

공항에서 비행기를 타려면 여러 검문소를 순서대로 통과해야 합니다. 신분증 확인 → 짐 검사 → 탑승구 확인. 각 검문소는 자기가 확인할 것만 확인하고, 문제가 없으면 다음 검문소로 그대로 보내줍니다. SecurityConfig와 JwtAuthenticationFilter가 만드는 구조도 이 공항 검문소와 똑같습니다.

① 프로젝트 코드 위치

loginauth/global/config/SecurityConfig.java +

loginauth/global/security/JwtAuthenticationFilter.java

② 해당 디자인 패턴

체인 오브 리스폰서빌리티 : 행동 패턴입니다. "여러 검문소를 사슬처럼 연결해두고, 각자 자기 역할만 확인한 뒤 다음으로 넘기는" 공항 보안 절차를 코드로 옮긴 것이 체인 오브 리스폰서빌리티 패턴입니다.

③ 코드 설명

```
// SecurityConfig.java
.addFilterBefore(
    jwtAuthenticationFilter,
    UsernamePasswordAuthenticationFilter.class
);

// JwtAuthenticationFilter.java
protected void doFilterInternal(HttpServletRequest request, HttpServletResponse response, FilterChain filterChain) {
    cookieService.readAccessToken(request).ifPresent(token -> { /* 토큰 검증, 인증 등록 */ });
    filterChain.doFilter(request, response); // 다음 검문소로 전달
}
```

SecurityConfig는 JwtAuthenticationFilter라는 검문소를

UsernamePasswordAuthenticationFilter라는 검문소 바로 앞자리에 세워둡니다. 이렇게 하면 모든 요청 (승객)은 반드시 JwtAuthenticationFilter를 먼저 통과한 뒤, 다음 검문소로 넘어가는 순서를 따르게 됩니다.

JwtAuthenticationFilter 내부를 보면, 자신이 맡은 검사 (쿠키에서 토큰 읽고, 검증하고, 인증 정보를 등록하는 것)만 마친 뒤, 마지막 줄에서 filterChain.doFilter (request, response)를 호출해 다음 검문소로 승객을 그대로 보내줍니다. 자기가 모든 검사를 다 하려 하지 않고, "내 역할만 확인하고 다음에게 넘기는" 것이 이 패턴의 핵심입니다.

④ 왜 필요했고, 왜 의미 있었는가?

인증 확인, 인가 확인, 로깅, CORS 처리처럼 통과해야 할 검문소가 여러 개일 때, 이걸 검문소 하나에서 전부 확인하려 든다면 그 검문소는 순식간에 아수라장이 될 겁니다.

각 검문소를 독립된 필터로 나누고 순서대로 연결해두면, 검문소 하나를 새로 추가하거나 순서를 바꾸거나 없애도 다른 검문소는 전혀 건드릴 필요가 없습니다. `addFilterBefore()` 한 줄로 "이 검문소를 여기에 세워달라"고 말하는 것만으로 전체 통과 순서가 재구성되는 것도 이 구조 덕분입니다.

정리하며

5개 패턴 모두 새로 배운 어려운 문법이 아닙니다. 복사기, 바리스타, 여행 패키지, 비서, 공항 검문소처럼 일상에서 흔히 겪는 장면들이 코드 속에 그대로 들어와 있었을 뿐입니다. 디자인 패턴은 "외워야 할 지식"이 아니라, "이미 잘 만들어진 코드에서 익숙한 이야기를 알아보는 눈"에 더 가깝습니다.

※ 다음 도메인, 다음 프로젝트를 만나더라도 "이 코드는 왜 이렇게 생겼을까"를 스스로 물어보시면, 오늘 만난 다섯 가지 이야기 중 하나로 설명되는 경우가 많을 것입니다.